Janne Törmänen
Haaga-Helia ammattikorkeakoulu
Linux-palvelimet - ICI003AS2A-3017

Raportti
3.4.2026

Tehtävä 2. Komentorivi: Johdanto ja tavoitteet

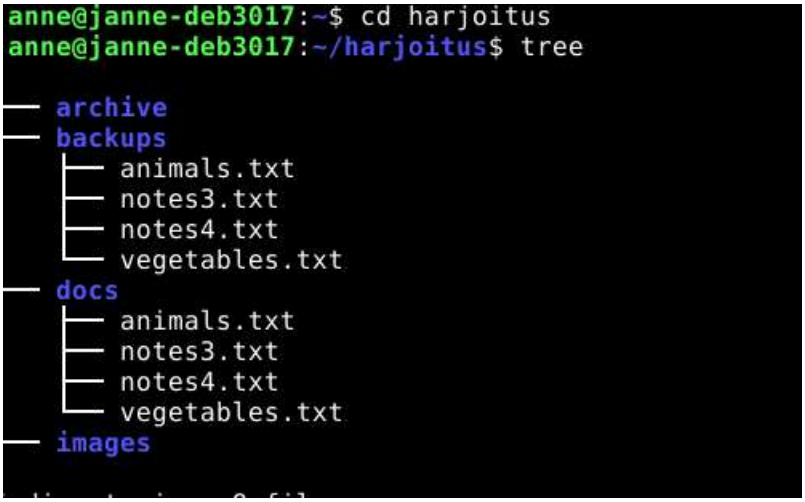
Tarkoituksena oli opetella komentorivin käyttöä, sekä opetella muutamia peruskomentoja, jolla voidaan hallita Linuxia. Tehtävinä oli luoda muun muassa: kotihakemistoon muutama kansio sekä niihin tiedostoja, *grepin* ja *pipen* käyttöä, Btop:in asennus ja yhden oman ohjelman asennus. Tiivistettynä tehtävän tarkoitus on päästä vähän tutuksi Linuxin komentorivin kanssa, sekä opetella käyttämään komentoja.

Tavoitteenani on ymmärtää, mitä erilaisia komentoja on olemassa, ja oppia käyttämään niitä oikein. Tiedän, että kaikkia komentoja ei voi muistaa, eikä tämä ole tarkoitus. Komentojen ymmärtäminen sekä uskallus kokeilla uusia komentoja on tärkeintä tällä hetkellä oman oppimisen kannalta.

Tehtävä 1. Peruskomennot

Tehtävässä 1. piti aluksi luoda kotihakemisto, jossa oli muutama kansio sekä ***docs*** kansion alla muutama tekstitiedosto. Kotihakemistoon luotiin kansio tähän komennolla ***mkdir*** *harjoitus*. Tässä piti kuitenkin olla tarkkana, että ollaan varmasti kotihakemistossa, jonka pystyi varmistaa, joko ***pwd*** komennolla tai ***tree*** jolla nähdään, missä olemme tällä hetkellä.

Cd*:llä** oli kätevä liikkua hakemistossa eri kansioitten välillä, jotta pystyi luomaan alikansioita sekä tiedostoja. Tiedostot luotiin komennolla ***nano notes1.txt ja niin edelleen. Tiedostoja pystyi muokkaamaan kirjoittamalla esimerkiksi ***nano notes1txt***, mutta piti varmistaa, että on oikeassa kansiossa, koska muuten se luo uuden tiedoston.



Kuva 1. *Harjoitus* hakemistorakenne tehtynä, sekä tiedostot luotuna ***docs*** kansioon, ja tehty varmuuskopiot ***backup*** kansioon.

Siirtäminen toiseen kansioon onnistui komennolla ***cp notes1.txt ~/harjoitus/backups***, mutta tässä piti olla tarkkana, että on kyseisessä kansiossa, muuten tämä komento ei toimi. Vaihtoehtoisesti voisi kirjoittaa koko polun alkuperä -> haluttu kohde.

Tehtävä 2. Grep and Pipe

Grep-komento etsii tiedostosta merkkirivejä, jotka esiintyvät tiedostossa. Tehtävän aloittaessa ihmettelin, miksi komennot ***grep apple fruits.txt*** ja ***grep -i apple fruits.txt*** tulostavat saman tekstin. Googlettelin hetken aikaa ja sain vastaukseksi, että lainausmerkit olivat erilaiset tehtävänannossa. Normaaleilla lainausmerkeillä tekstit menevät eri riville, kun taas tehtävänannossa teksti on yhdessä rivissä. Tässä vielä merkitykset näille testille:

- **grep apple fruits.txt**: Tässä tapauksessa etsii sanan ”apple” tiedostosta, mutta pelkästään pienellä kirjaimella alkavat apple sanat.
- **grep -i apple fruits.txt**: Sama periaate, mutta ei välitä siitä alkaako sana pienellä vai isolla kirjaimella.
- **grep -n apple fruits.tx**: kertoo numerolla, monennellako rivillä apple sijaitsee. Tässä tapauksessa 1: apple
- **grep -ni apple fruits.txt**: Ei ole enään tarkka kirjaimen koosta, joten tulostus on 1: apple 4: Apple pie
- **grep -v apple fruits.txt**: Tulostaa kaiken muun tiedosta, paitsi apple

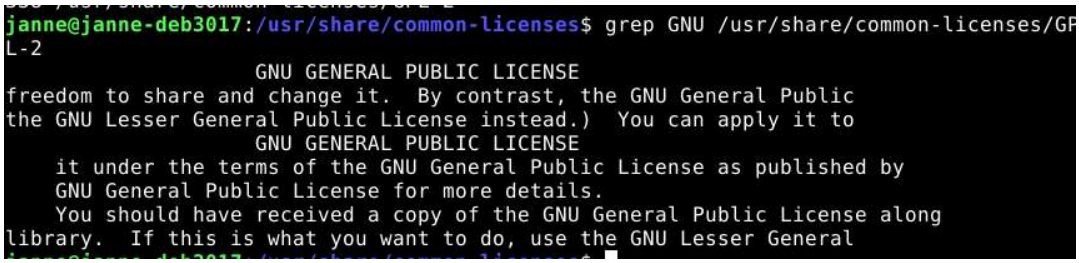
Seuraavana olisi tarkoitus tutkia ***PIPEä*** ja miten se toimii. **Pipe**-komento Linuxissa toimii siten, että pystyt ketjuttamaan kahta tai useampaa komentoa yhdessä komenossa. (Carter 2024).

Tarkoitus oli luoda tiedosto, ja lähteä tutkimaan, miten pipe-komennot toimivat tässä tapauksessa.

- **cat animals.txt | grep cat**: Ensinmäinen syöte ohjaa suoraan tiedostoon ja PIPEn jälkeen tulostaa sanat, jotka alkavat sanalla *cat.*
- **cat animals.txt | wc -l:** Ensinmäinen syöte ohjaa suoraan tiedostoon ja PIPEn jälkeen tulostaa määrällisesti, montako sanaa on tiedostossa yhteensä.
- **cat animals.txt | sort | uniq**: Ensinmäinen syöte ohjaa suoraan tiedostoon ja PIPEn jälkeen lajittelee tiedosto aakkosjärjestykseen ja kolmannessa PIPEssä pitäisi poistaa ylimääräiset duplikaatit (sana, joka sisältyy kahdesti samassa tiedostossa), mutta tässä ei ole niitä, joten se ei toteudu (GeekforGeeks 2025).

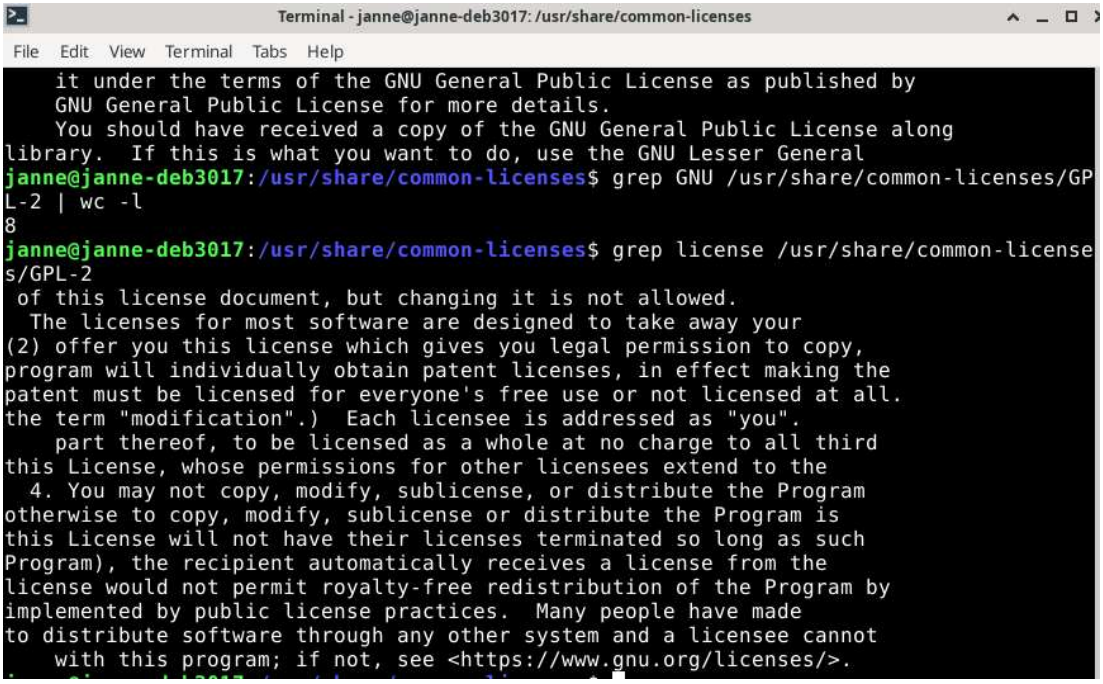
Viimeisessä tehtävässä oli tarkoitus testata *grepin*, *pipen* ja *wc* komentojen käyttöä. Tehtävässä lähdettiin selvittämään eri komennoilla, mitä tiedosto sisältää tai mitä tapahtuu eri komennoilla.

- **How many lines do GPL-2 license file contain?** = 338
- **What does command grep GNU /usr/share/common-licenses/GPL-2 do?** = Näyttää ne rivit ainoastaan, jossa sijaitsee sana GNU.



Kuva 2. GREPillä GNU tulokset riveistä, joista tämä sana löytyy.

- Count how many lines in GPL-2 license file contain the word “GNU” = Vastaus on, että sana GNU esiintyy kahdeksalla rivillä yhteensä. Tämä saatiin selvitettyä *grep GNU /usr/share/common-licenses/GPL-2 | wc -l*
- Show all lines that include word “license”:



Kuva 3. Tässä on rivit, josta löytyy sana *license*. Komento oli tämä: grep license /usr/share/common-licenses/GPL-2

- Linux is case sensitive. The previous command lists all lines that include word license written in small letters. How to make this command case insensitive?: Laittamalla grepin jälkeen “**-i**”, tämä komento näyttää kaikki *license* sanat, vaikka se alkaisi isolla kirjaimella.
- Make your own query that uses grep, pipe and/or wc and explain what it does : ***cat GPL-2 | wc -w***, tässä siis ensimmäisessä vaiheessa menemme suoraan tiedostoon ja seuraavaksi laskemme, montako sanaa tässä tiedostossa on.

Tehtävä 3. BTOP

Btop on monitori, joka näyttää reaaliajassa statistiikkaa muun muassa prosessorista, muistin käytöstä, kovalevyn tilasta. Ohjelman asennuksen jälkeen pääsin katsomaan, miltä se näyttää. Tässä on myös muutamaan tehtävään vastaus:

- Where do binaries go? check with command which btop: Tiedoston polku on komennolla ***which btop,/usr/bin/btop***.

Testaamalla myös komentoa ***dpkg -L btop*** saatiin näytöllemme kaikki kansiot sekä tiedostot, mitä btop-paketti asensi meidän järjestelmäämme. Tehtävässä piti myös luoda varmuuskopio tiedostosta, ja se piti myös tallentaa samaan hakemistoon, tämä onnistui tällä komennolla:

```
janne@janne-deb3017:~$ cp ~/.config/btop/btop.conf ~/.config/btop/btop.conf.ori
janne@janne-deb3017:~$
```

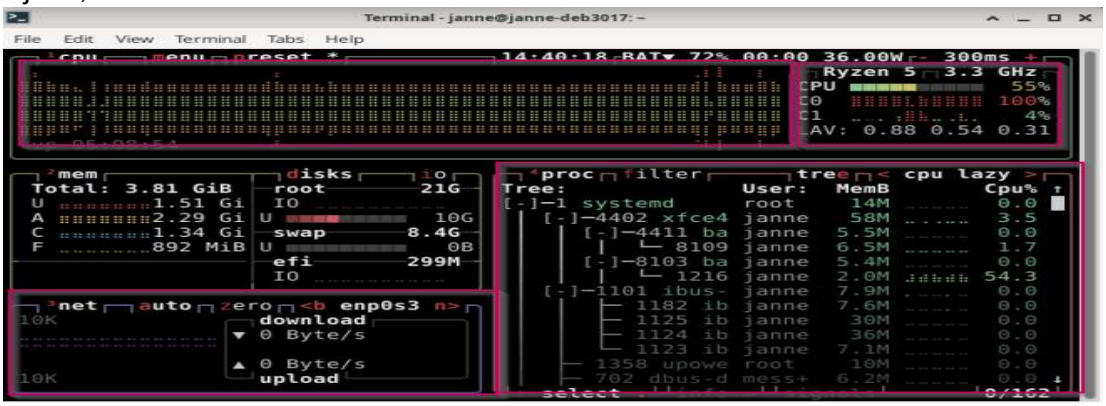
Kuva 4. Ensimmäinen polku on alkuperäinen tiedosto ja toinen polku on tiedostolle uusi nimi, joka sijaitsee samassa hakemistossa.

Kun meillä oli varmuuskopio tiedostosta, lähdimme muokkaamaan alkuperäistä tiedostoa ja tarkastelemaan, mitä tapahtuu tämän jälkeen. Tässä vielä kuva muutoksistani:

```
  GNU nano 8.4                    btop.conf *
graph_symbol_net = "default"

# Graph symbol to use for graphs in cpu box, "default", "braille", "block" or "
graph_symbol_proc = block"

#* Manually set which boxes to show. Available values are "cpu mem net proc" an
shown_boxes = "cpu mem net proc"

#* Update time in milliseconds, recommended 2000 ms or above for better sample
update_ms = 300

#* Processes sorting, "pid" "program" "arguments" "threads" "user" "memory" "cp
#* "cpu lazy" sorts top process over time (easier to follow), "cpu direct" upda
proc_sorting = "cpu lazy"

#* Reverse sorting order, True or False.
proc_reversed = False
#* Show processes as a tree.
proc_tree = Truuuuuue

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Kuva 5. Nanossa tehdyt muutokset ohjeiden mukaisesti, ja tein myös yhden oman muutoksen vaihtamalla **graph_symbol_proc** tilaan **block**.

Tämän jälkeen huomasin avatessani btop:in, päivtystiedot näkyivät huomattavasti nopeammin 3000ms -> 300ms, sekä proressit näkyvät puumaisessa hierarkiassa listan sijaan, tässä vielä kuva:



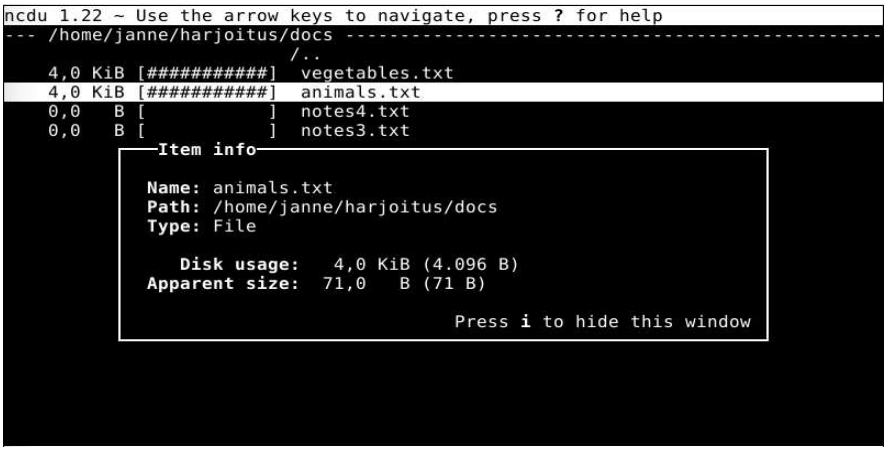
Kuva 6. Ylhäällä päivitysnopeus tihentyi sekä käytin komennolla ***yes > /dev/null*** yhden prosessorini kokonaan. Myös pingaamalla sain downloadiin ja uploadiin dataa, mutta

tässä sitä ei näy prosessorin kuormittamisen takia. Myös puumainen rakenne on tullut nano muutoksen myötä.

4. Install Your Own Command-Line Application

Tässä tehtävässä lähdin asentamaan itselleni ncdu-sovellusta. Tämän sovellus tarjoaa helppokäyttöisen tavan tarkastella tiedostoja, niiden polkuja sekä paljonko nämä vievät levytilaa. (Stocker 2023).

Kun asensin sovelluksen ja lähdin kokeilemaan, miten tämä muuttaisi helppokäyttöisemmäksi tiedostojen tarkastelun sekä mitä muuta tämä tarjoaisi. Huomasin hetken ajan käytettyäni, että sovellus antaa hyvin tietoa siitä, missä tiedostoissa on dataa, mitkä ovat tyhjiä, sekä näyttää polun suoraan tähän tiedostoon/kansioon. Tämän polun myös pystyy hyvin kopiomaan suoraan, joka ainakin tällein aluksi helpottaa itselle oikean polun löytämistä. Sovelluksen parametrejä en päässyt muokkaamaan. Tässä kuva, jossa näytän harjoituskansion tiedoston tiedot.



Kuva 7. ncdu-sovellus ja sen käyttäminen. Tässä näkyy, kuinka olen päässyt näkemään esimerkkisi animals-tekstitiedoston tiedot.

Yhteenveto

Tehtävä oli itselle todella opettavainen ja jossain määrin myös työläs. En ole ennen käyttänyt Linuxia, joten komentojen opettelu vei jonkun verran aikaa sekä niiden ymmärtäminen oli haastavaa. Kuitenkin päivän päätteeksi opin yllättävän hyvin käyttämään komentoja ja luulen, että nämä selkeentyvät vielä paremmin, kun käytämme komentoja enemmän muun muassa Apache2-asennuksen yhteydessä. Koen myös, että komennoissa on yllättävän paljon loogisia päätelmiä esimerkiksi *mkdir* tulee sanoista *makedirectory*.

Oli kivaa oppia luomaan kansioita, tiedostoja sekä muokata näitä nanossa ja asentamaan sovelluksia. Tehtävän tavoitteisiin olen mielestäni päässyt hyvin ja täytin omat odotukset, tästä on hyvä jatkaa eteenpäin kohti uusia tehtäviä.

Lähteet

Emily, C. 22.6.2024. Mikä on putki Linuxissa?. Pipe-, Grep- ja Sort-komento Linuxissa/Unixissa esimerkein. Luettavissa: https://www.guru99.com/fi/linux-pipe-grep.html. Luettu: 1.4.2026

Geekforgeeks 2025. Uniq Command in Linux with Examples**.** Luettavissa: https://www.geeksforgeeks.org/linux-unix/uniq-command-in-linux-with-examples/. Luettu: 1.4.2026

Manpages 2026. Manpages. Luettavissa: https://man.cx/grep(1)/fi. Luettu: 1.4.2026

Sandra, S. 11.4.2023. Using the Linux ncdu command to view your disk usage. Unix as a Second Language. Luettavissa: https://www.networkworld.com/article/972132/using-the-linux-ncdu-command-to-view-your-disk-usage.html. Luettu: 3.4.2026